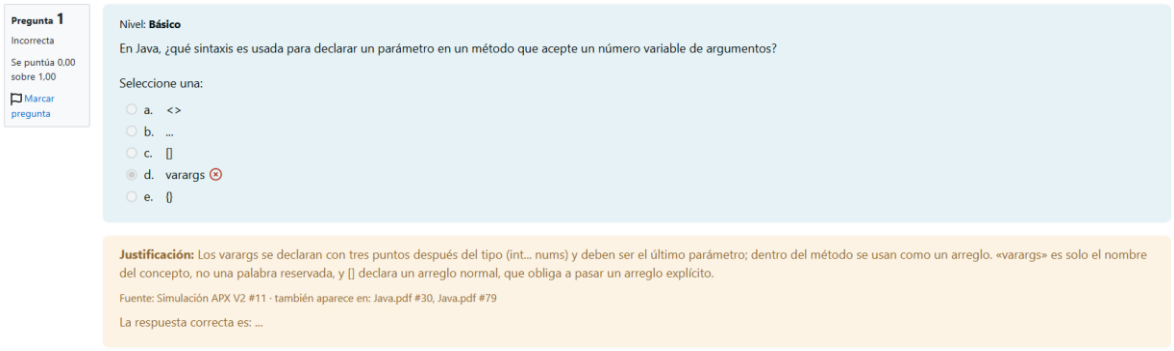
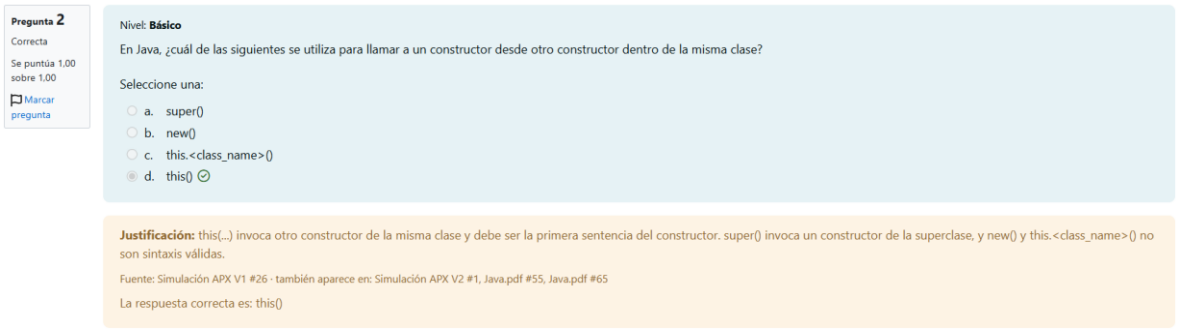


Mi justificación
Este no sabía, pensé que varargs era una expresión para “various arguments” o así



Mi justificación
El comando this() permite comunicarse a los constructores dentro de una clase, aun si están en diferentes. De esta forma se pueden cambiar los atributos de la clase entre varios constructores.



Pregunta 3

Incorrecta

Se puntúa 0,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

Which class has a default constructor?

```
class X{}
class Y{
    Y(){}
}
class Z{
    Z(int i){}
}
```

Seleccione una:

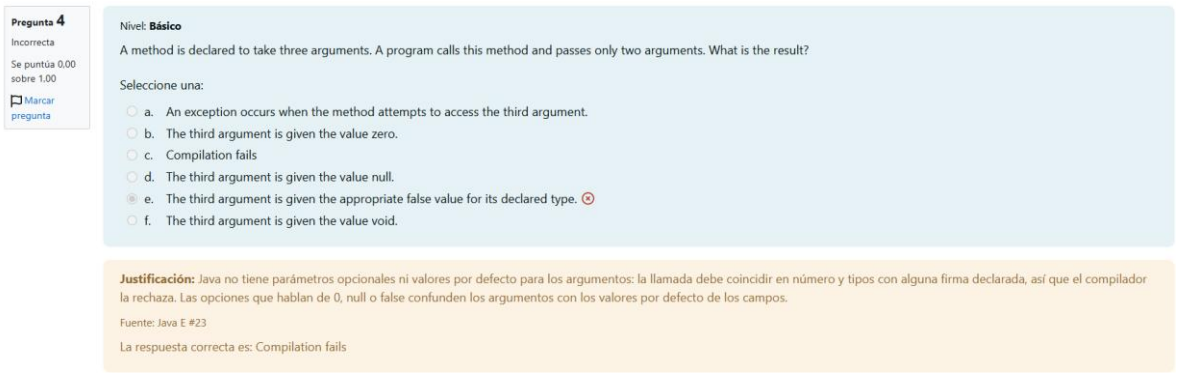
- a. X and Y
- b. Y only
- c. X and Z
- d. Y and Z ⊗
- e. X, Y and Z
- f. Z only.
- g. X only.

Justificación: El constructor por defecto es el que genera el compilador cuando la clase no declara ningún constructor: solo X lo tiene. Y declara explícitamente un constructor sin argumentos, que es equivalente pero no es el «default» en sentido estricto. Z solo tiene Z(int), así que ni siquiera tiene constructor sin argumentos.

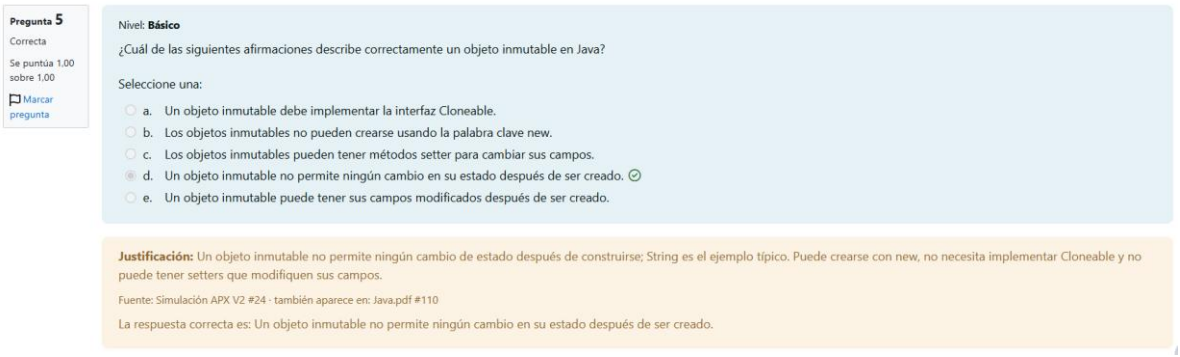
Fuente: Java DE #18

La respuesta correcta es: X only.

Mi justificación
Las clases Y y Z tienen un constructor ya que dentro de Y está el constructor Y() y dentro de Z está el constructor Z(int i) (No leí la palabra default).



Mi justificación
Si se requieren 3 argumentos pero sólo se dan 2, el tercero será el equivalente a falso o null si es un objeto.



Mi justificación
Un objeto inmutable no permite ningún cambio en su estado después de ser creado, es teoría básica. Un ejemplo son los String.



Pregunta 6

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

What is the result?

```
class X {
    String str = "default";
    X(String s) { str = s; }
    void print() { System.out.println(str); }
    public static void main(String[] args) { new X("hello").print(); }
}
```

Seleccione una:

- a. hello
- b. default
- c. Compilation fails.
- d. The program prints nothing.
- e. An exception is thrown at run time.

Justificación: Primero se ejecuta el inicializador del campo (str = "default") y después el cuerpo del constructor, que lo sobrescribe con "hello". print() imprime ese valor final. El distractor «default» ignora que el cuerpo del constructor corre después de los inicializadores de instancia.

Fuente: Java Sesiones #9 · también aparece en: Quiz Desarrollo APX 1 #14

La respuesta correcta es: hello

Mi justificación
La clase X tiene un constructor que ocupa un string s como parámetro. Al hacer new X(“hello”) creamos una instancia en la que el valor de su atributo str será ahora s. Entonces al hacer print se imprime este nuevo valor de str que es “hello”.



Pregunta 7

Correcta

Se puntúa 1.00 sobre 1.00

Marcar pregunta

Nivel: **Básico**

What is the result?

```
class X {
    static void m(int i){
        i+=7;
    }
    public static void main(String[] args){
        int i = 12;
        m(i);
        System.out.println(i);
    }
}
```

Seleccione una:

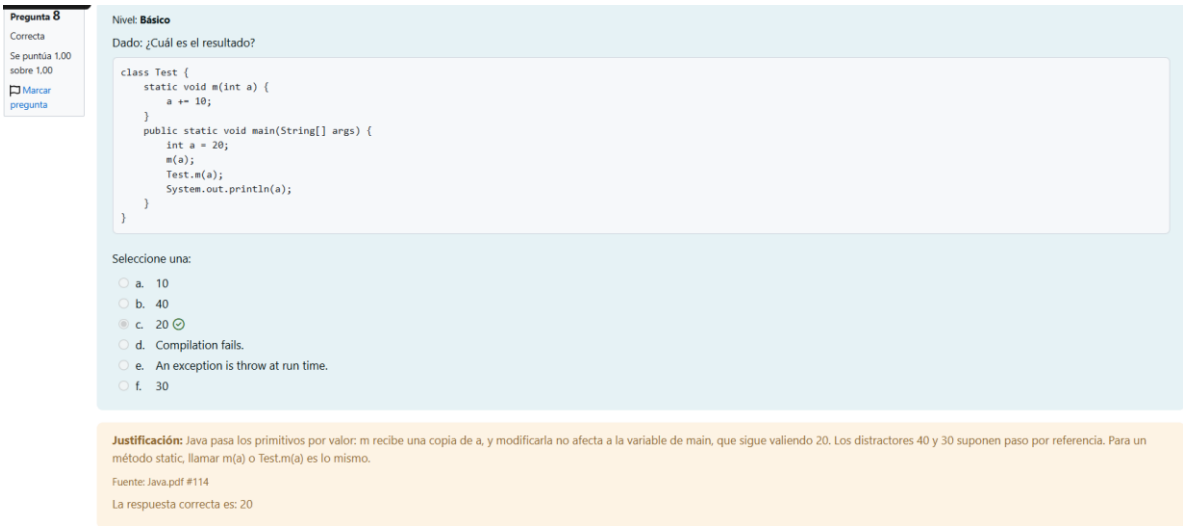
a. Compilation fails.
b. 12
c. 7
d. 19
e. An exception is thrown at run time.

Justificación: Java pasa los argumentos por valor: m() modifica su copia local de i, y la variable de main sigue valiendo 12. El distractor 19 supone paso por referencia.

Fuente: Java D #14 · también aparece en: Java Sesiones #30

La respuesta correcta es: 12

Mi justificación
En la clase X, no se declaro explícitamente el atributo i, así que i solo existe como parámetro de m() y por lo tanto el scope indica que solo existe dentro de i. El scope del println() apunta a la i = 12, no a la i dentro de m(), así que da 12



Pregunta 8

Correcta

Se puntúa 1.00 sobre 1.00

Marcar pregunta

Nivel: **Básico**

Dado: ¿Cuál es el resultado?

```
class Test {
    static void m(int a) {
        a += 10;
    }
    public static void main(String[] args) {
        int a = 20;
        m(a);
        Test.m(a);
        System.out.println(a);
    }
}
```

Seleccione una:

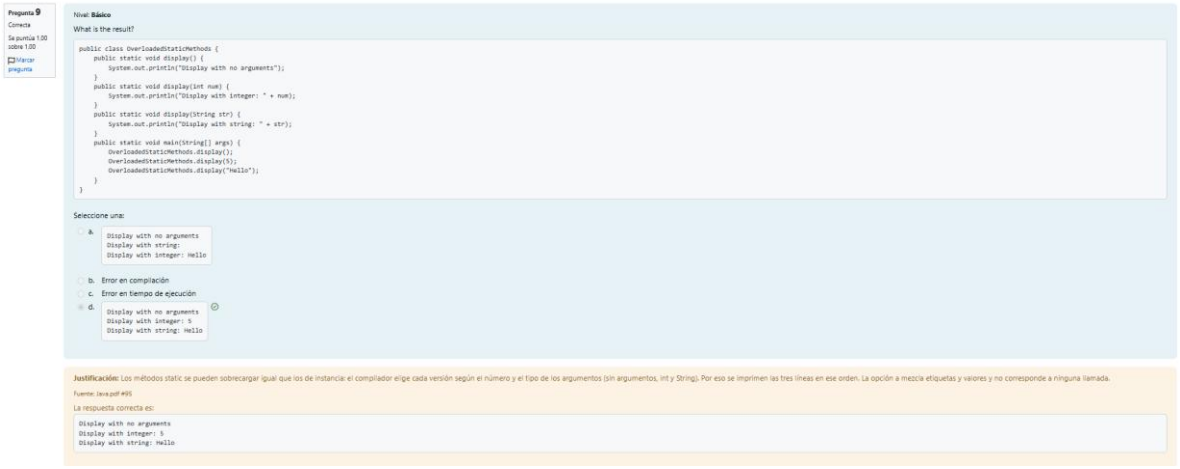
a. 10
b. 40
c. 20 ✓
d. Compilation fails.
e. An exception is throw at run time.
f. 30

Justificación: Java pasa los primitivos por valor: m recibe una copia de a, y modificarla no afecta a la variable de main, que sigue valiendo 20. Los distractores 40 y 30 suponen paso por referencia. Para un método static, llamar m(a) o Test.m(a) es lo mismo.

Fuente: Java.pdf #114

La respuesta correcta es: 20

Mi justificación
Lo mismo que la pregunta anterior, la a de m(a) y la de Test.m(a) solo existe dentro de la función, pero el valor impreso será el de a = 20 en este caso.



Pregunta
Correcta
Se puntúa 1,00 sobre 1,00
Marcar pregunta

Nivel: **Básico**

What is the result?

```
public class OverloadedStaticMethods {
    public static void display() {
        System.out.println("Display with no arguments");
    }
    public static void display(int num) {
        System.out.println("Display with integer: " + num);
    }
    public static void display(String str) {
        System.out.println("Display with string: " + str);
    }
    public static void main(String[] args) {
        OverloadedStaticMethods.display();
        OverloadedStaticMethods.display(5);
        OverloadedStaticMethods.display("Hello");
    }
}
```

Seleccione una:

a.
```
Display with no arguments
Display with string:
Display with integer: Hello
```

b. Error en compilación

c. Error en tiempo de ejecución

d.
```
Display with no arguments
Display with integer: 5
Display with string: Hello
```

Justificación: Los métodos static se pueden sobrecargar igual que los de instancia: el compilador elige cada versión según el número y el tipo de los argumentos (sin argumentos, int y String). Por eso se imprimen las tres líneas en ese orden. La opción a mezcla etiquetas y valores y no corresponde a ninguna llamada.

Fuente: Java.pdf #95

La respuesta correcta es:

```
Display with no arguments
Display with integer: 5
Display with string: Hello
```

Mi justificación
Bastante directa, se ejecutan los 3 métodos de la clase que imprimen texto sin argumentos, texto con un número y texto con un String. Ya que los 3 métodos son static, no necesitan instanciarse así que está bien.



Pregunta 10

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

What will make this code compile and run?

```
01. public class Simple {
02.
03.     public float price;
04.     public static void main(String[] args) {
05.
06.         Simple price = new Simple();
07.         price = 4;
08.     }
09. }
```

Seleccione una:

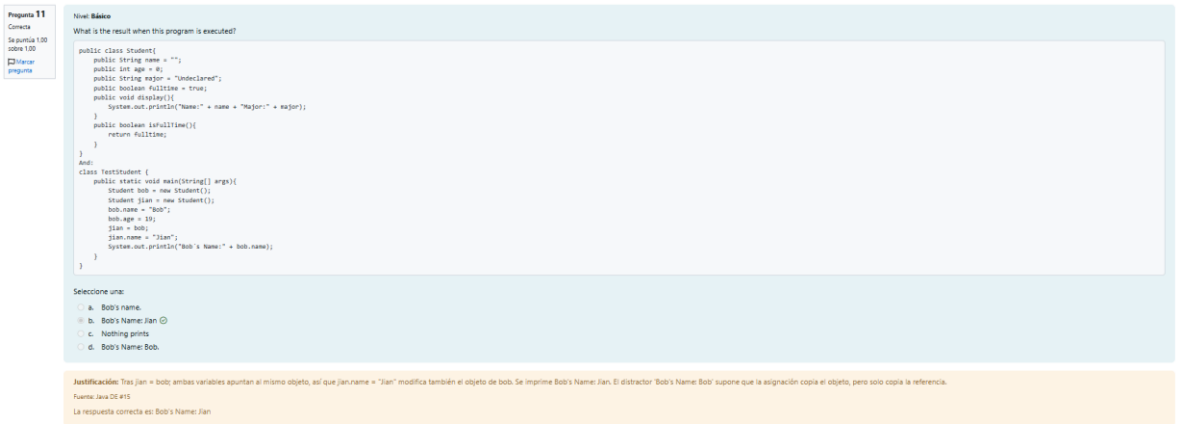
a. Change line 3 to the following: public int price;
b. Change line 7 to the following: int price = new Simple();
c. Change line 7 to the following: float price = new Simple ();
d. Change line 7 to the following: price = 4f;
e. Change line 7 to the following: price.price = 4;

Justificación: La variable local price es de tipo Simple, así que asignarle 4 no compila. Con price.price = 4 se asigna el int 4 al campo float del objeto, que es una conversión de ampliación permitida. Cambiar la línea 3 no arregla nada, porque el error está en la línea 7. Las opciones b, c y d intentan mezclar un Simple con un tipo primitivo.

Fuente: Java Sesiones #16 · también aparece en: Java E #15

La respuesta correcta es: Change line 7 to the following: price.price = 4;

Mi justificación
Una inicialización de un valor boleano requiere que sea true, false, o una sentencia lógica que devuelva alguno de los dos anteriores. Bajo esta lógica: XXX a) y c) porque es una operación aritmética, no una comparación XXX b) porque “true” es un String VVV d), e) y f) son comparaciones de números.



Pregunta 11
Correcta
Se puntúa 1,00 sobre 1,00
Marcar pregunta

Nivel: Básico

What is the result when this program is executed?

```
public class Student{
    public String name = "";
    public int age = 0;
    public String major = "Undeclared";
    public boolean fulltime = true;
    public void display(){
        System.out.println("Name:" + name + "Major:" + major);
    }
    public boolean isFullTime(){
        return fulltime;
    }
}
And:
class TestStudent {
    public static void main(String[] args){
        Student bob = new Student();
        Student jian = new Student();
        bob.name = "Bob";
        bob.age = 19;
        jian = bob;
        jian.name = "Jian";
        System.out.println("Bob's Name:" + bob.name);
    }
}
```

Seleccione una:

a. Bob's name.
b. Bob's Name: Jian ✓
c. Nothing prints
d. Bob's Name: Bob.

Justificación: Tras jian = bob; ambas variables apuntan al mismo objeto, así que jian.name = "Jian" modifica también el objeto de bob. Se imprime Bob's Name: Jian. El distractor 'Bob's Name: Bob' supone que la asignación copia el objeto, pero solo copia la referencia.

Fuente: Java DE #15

La respuesta correcta es: Bob's Name: Jian

Mi justificación
Al hacer jian = bob estamos cambiando la referencia de jian a la de bob, ahora al alterar los atributos de jian también alterará los de bob, entonces jian.name = “Jian” cambia también el de bob y por tanto bob.name es Jian

Pregunta 12

Correcta

Se puntúa 1.00 sobre 1.00

Marcar pregunta

Nivel: **Básico**

What is the result?

```
public class Barn{
    public static void main(String[] args){
        new Barn().go("hi", 1);
        new Barn().go("hi", "world", 2);
    }
    public void go(String... y, int x){
        System.out.print(y[y.length - 1] + " ");
    }
}
```

Seleccione una:

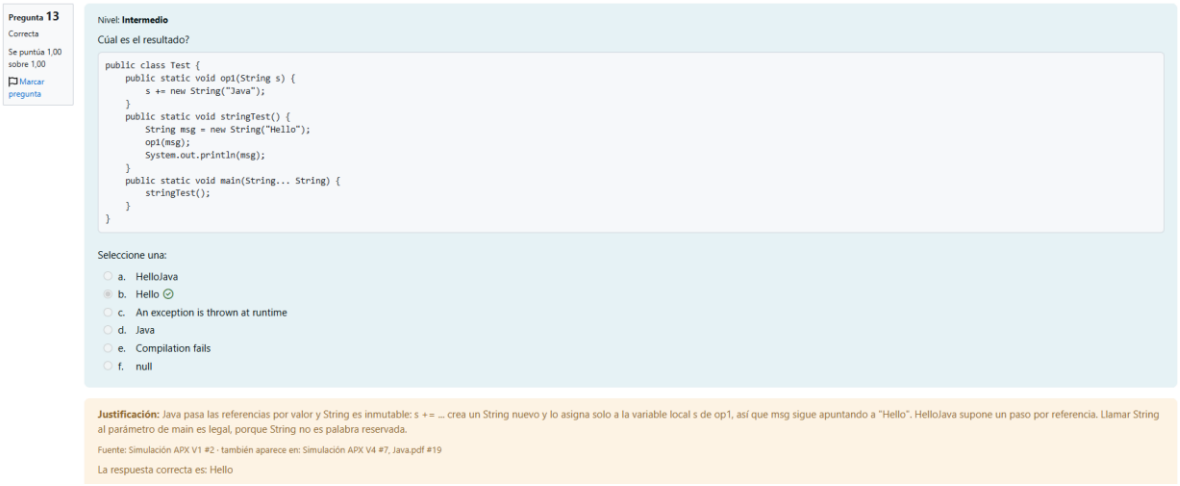
- a. hi world
- b. Compilation fails ✓
- c. world world.
- d. An exception is thrown at runtime.
- e. hi hi

Justificación: Un parámetro varargs tiene que ser el último de la lista, así que go(String... y, int x) es ilegal y la clase no compila. 'hi world' es lo que se imprimiría si la firma fuera go(int x, String... y).

Fuente: Java DE #8

La respuesta correcta es: Compilation fails

Mi justificación
Está la respondí al azar pero supuse que era fallo en compilación.



Pregunta 13
Correcta
Se puntúa 1,00 sobre 1,00
Marcar pregunta

Nivel: **Intermedio**

Cúal es el resultado?

```
public class Test {
    public static void op1(String s) {
        s += new String("Java");
    }
    public static void stringTest() {
        String msg = new String("Hello");
        op1(msg);
        System.out.println(msg);
    }
    public static void main(String... String) {
        stringTest();
    }
}
```

Seleccione una:

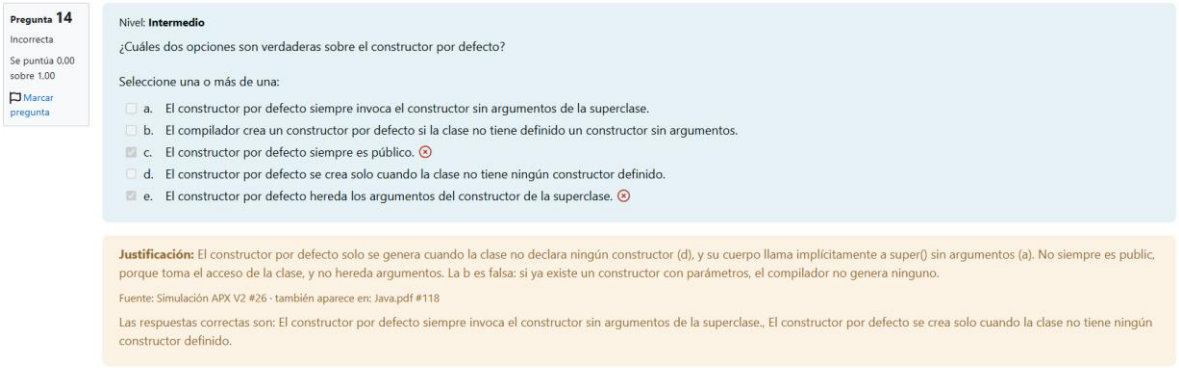
a. HelloJava
b. Hello
c. An exception is thrown at runtime
d. Java
e. Compilation fails
f. null

Justificación: Java pasa las referencias por valor y String es inmutable: s += ... crea un String nuevo y lo asigna solo a la variable local s de op1, así que msg sigue apuntando a "Hello". HelloJava supone un paso por referencia. Llamar String al parámetro de main es legal, porque String no es palabra reservada.

Fuente: Simulación APX V1 #2 · también aparece en: Simulación APX V4 #7, Java.pdf #19

La respuesta correcta es: Hello

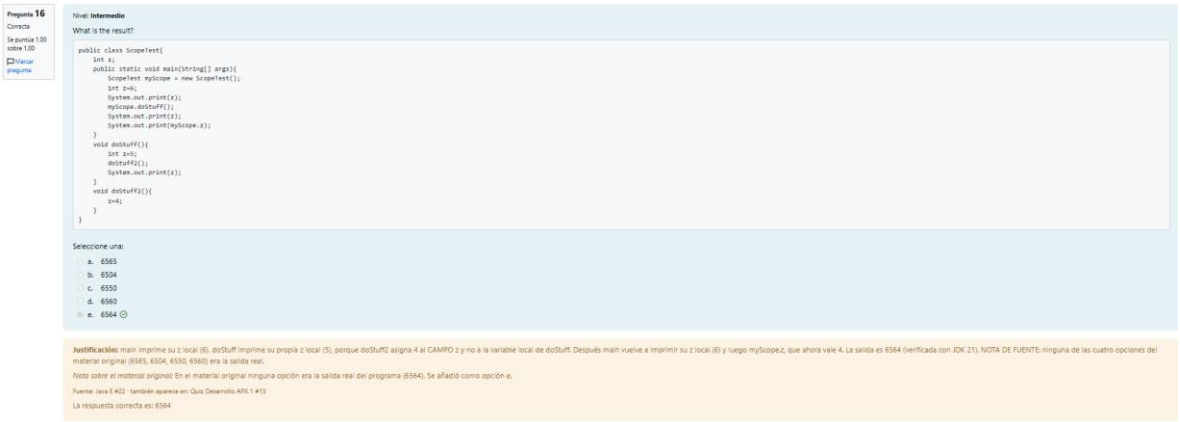
Mi justificación
String es inmutabe, así que op1(msg) sí hace la operación de concatenar “Hello” y “Java” pero no la guarda en un atributo ni la retorna, así que el print imprime el msg original que era “Hello”



Mi justificación
No sabía cuál era el constructor por defecto, la respondí al azar.



Mi justificación
Leyendo bien, vemos que las tres correctas son class2 has a v2, class3 has a v1 y class2 has a class1.



Pregunta 16
Correcta
Se puntúa 1.00 sobre 1.00
Marcar pregunta

Nivel: **Intermedio**

What is the result?

```
public class ScopeTest{
    int z;
    public static void main(String[] args){
        ScopeTest myScope = new ScopeTest();
        int z=6;
        System.out.print(z);
        myScope.doStuff();
        System.out.print(z);
        System.out.print(myScope.z);
    }
    void doStuff(){
        int z=5;
        doStuff2();
        System.out.print(z);
    }
    void doStuff2(){
        z=4;
    }
}
```

Seleccione una:

a. 6565
b. 6504
c. 6550
d. 6560
e. 6564

Justificación: main imprime su z local (6). doStuff imprime su propia z local (5), porque doStuff2 asigna 4 al CAMPO z y no a la variable local de doStuff. Después main vuelve a imprimir su z local (6) y luego myScope.z, que ahora vale 4. La salida es 6564 (verificada con JDK 21). NOTA DE FUENTE: ninguna de las cuatro opciones del material original (6565, 6504, 6550, 6560) era la salida real.

Nota sobre el material original: En el material original ninguna opción era la salida real del programa (6564). Se añadió como opción e.

Fuente: Java E #22 · también aparece en: Quiz Desarrollo APX 1 #13

La respuesta correcta es: 6564

Mi justificación
El primer print imprime el z declarado justo en la línea anterior, el cual z = 6. La siguiente línea ejecuta doStuff() el cual imprime el z = 5 porque se declaró dentro de la función, así que reemplaza al z que se incializó al inicio de la clase. El siguiente print imprime otra vez z = 6. El último print imprime z = 4 porque en doStuff() se ejecutó doStuff2() que hace z = 4 y en este SÍ cambia el atributo z ya que no lo declaró dentro de la función. Al final se imprimió 6564.



Pregunta 17
Incorrecta
Se puntúa 0.00 sobre 1.00
Marcar pregunta

Nivel: **Intermedio**

What is the result?

```
public class ScopeTest{
    int x;
    public static void main(String[] args){
        ScopeTest myScope = new ScopeTest();
        int z = 6;
        System.out.println(z);
        myScope.doStuff();
        System.out.print(z);
        System.out.print(myScope.z);
    }
    void doStuff(){
        int z = 5;
        doStuff2();
        System.out.print(myScope.z);
    }
    void doStuff2(){
        z = 4;
    }
}
```

Seleccione una:

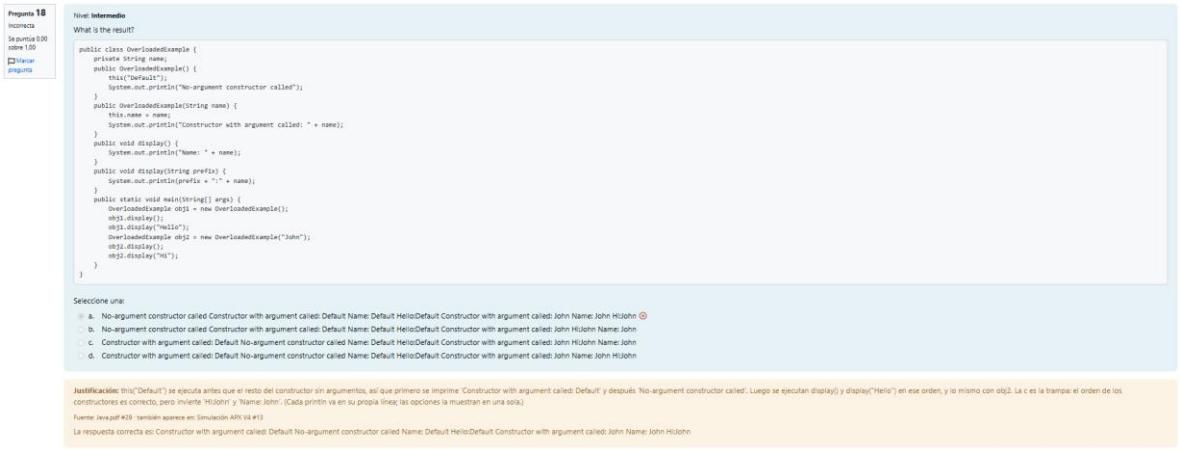
a. Compilation fails
b. 6564 ✗
c. 6565
d. 6554
e. 6566

Justificación: myScope es una variable local de main, así que no existe dentro de doStuff(); myScope.z en ese método no compila. Además, a la declaración de main le falta el '=', que es otro error de compilación. Los distractores numéricos (6564, 6554...) suponen que el código compila y solo siguen el sombreado de z.

Fuente: Java OE #7

La respuesta correcta es: Compilation fails

Mi justificación
Pensé que era el mismo ejercicio, pero después vi que doStuff() está diferente.



Pregunta 18
Incorrecta
Se puntúa 0.00 sobre 1.00
Marcar pregunta

Nivel: **Intermedio**

What is the result?

```
public class OverloadedExample {
    private String name;
    public OverloadedExample() {
        this("Default");
        System.out.println("No-argument constructor called");
    }
    public OverloadedExample(String name) {
        this.name = name;
        System.out.println("Constructor with argument called: " + name);
    }
    public void display() {
        System.out.println("Name: " + name);
    }
    public void display(String prefix) {
        System.out.println(prefix + ":" + name);
    }
    public static void main(String[] args) {
        OverloadedExample obj1 = new OverloadedExample();
        obj1.display();
        obj1.display("Hello");
        OverloadedExample obj2 = new OverloadedExample("John");
        obj2.display();
        obj2.display("Hi");
    }
}
```

Seleccione una:

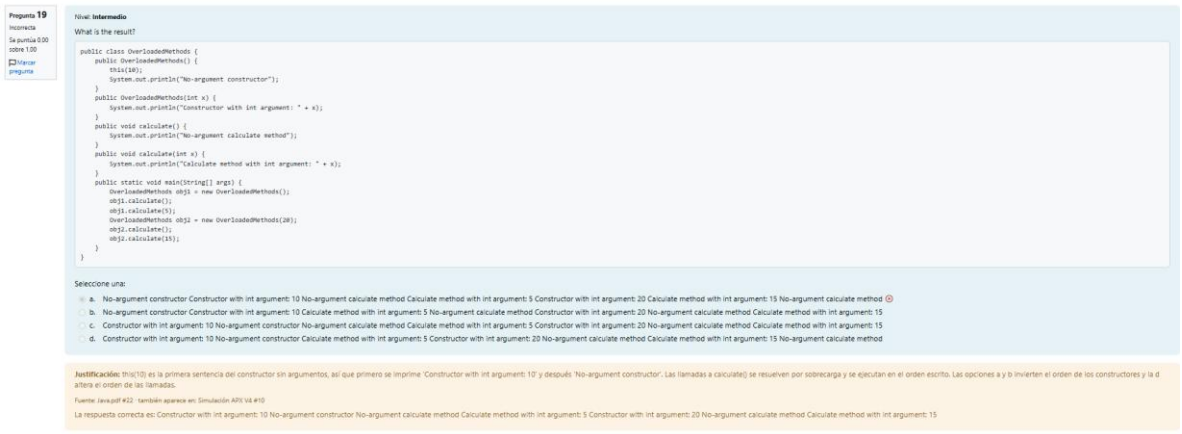
a. No-argument constructor called Constructor with argument called: Default Name: Default Hello:Default Constructor with argument called: John Name: John Hi:John
b. No-argument constructor called Constructor with argument called: Default Name: Default Hello:Default Constructor with argument called: John Hi:John Name: John
c. Constructor with argument called: Default No-argument constructor called Name: Default Hello:Default Constructor with argument called: John Hi:John Name: John
d. Constructor with argument called: Default No-argument constructor called Name: Default Hello:Default Constructor with argument called: John Name: John Hi:John

Justificación: this("Default") se ejecuta antes que el resto del constructor sin argumentos, así que primero se imprime 'Constructor with argument called: Default' y después 'No-argument constructor called'. Luego se ejecutan display() y display("Hello") en ese orden, y lo mismo con obj2. La c es la trampa: el orden de los constructores es correcto, pero invierte 'Hi:John' y 'Name: John'. (Cada println va en su propia línea; las opciones la muestran en una sola.)

Fuente: Java.pdf #29 · también aparece en: Simulación API: V4 #13

La respuesta correcta es: Constructor with argument called: Default No-argument constructor called Name: Default Hello:Default Constructor with argument called: John Name: John Hi:John

Mi justificación
La primera línea crea la instancia obj1 así que ejecuta e imprime el constructor sin argumentos. El constructor de la primera línea de main ejecuta this(“Default”) así que ejecuta e imprime el constructor con argument called: Degault. La línea obj1.display() imprime el nombre Name:Default La línea obj1.display(“Hello”) imprime Hello:Default. La siguiente línea crea la instancia obj2 pero esta vez ejecuta el constrctor con argumento “John”. La línea obj2.display() imprime el nombre Name:John La línea obj2.display(“Hello”) imprime Hi:John Agrupando todo queda la a)



Pregunta 19
Incorrecta
Se puntúa 0.00 sobre 1.00
Marcar pregunta

Nivel: **Intermedio**

What is the result?

```
public class OverloadedMethods {
    public OverloadedMethods() {
        this(10);
        System.out.println("No-argument constructor");
    }
    public OverloadedMethods(int x) {
        System.out.println("Constructor with int argument: " + x);
    }
    public void calculate() {
        System.out.println("No-argument calculate method");
    }
    public void calculate(int x) {
        System.out.println("Calculate method with int argument: " + x);
    }
    public static void main(String[] args) {
        OverloadedMethods obj1 = new OverloadedMethods();
        obj1.calculate();
        obj1.calculate(5);
        OverloadedMethods obj2 = new OverloadedMethods(20);
        obj2.calculate();
        obj2.calculate(15);
    }
}
```

Seleccione una:

a. No-argument constructor Constructor with int argument: 10 No-argument calculate method Calculate method with int argument: 5 Constructor with int argument: 20 Calculate method with int argument: 15 No-argument calculate method ✗
b. No-argument constructor Constructor with int argument: 10 Calculate method with int argument: 5 No-argument calculate method Constructor with int argument: 20 No-argument calculate method Calculate method with int argument: 15
c. Constructor with int argument: 10 No-argument constructor No-argument calculate method Calculate method with int argument: 5 Constructor with int argument: 20 No-argument calculate method Calculate method with int argument: 15
d. Constructor with int argument: 10 No-argument constructor Calculate method with int argument: 5 Constructor with int argument: 20 No-argument calculate method Calculate method with int argument: 15 No-argument calculate method

Justificación: this(10) es la primera sentencia del constructor sin argumentos, así que primero se imprime 'Constructor with int argument: 10' y después 'No-argument constructor'. Las llamadas a calculate() se resuelven por sobrecarga y se ejecutan en el orden escrito. Las opciones a y b invierten el orden de los constructores y la d altera el orden de las llamadas.

Fuente: Java.pdf #22 también aparece en: Simulación API: V4 #10

La respuesta correcta es: Constructor with int argument: 10 No-argument constructor No-argument calculate method Calculate method with int argument: 5 Constructor with int argument: 20 No-argument calculate method Calculate method with int argument: 15

Mi justificación
Al crear la instancia obj1 se ejecuta e imprime el constructor sin argumentos. Ese constructor ejecuta otro constructor con this(10) así que ejecuta e imprime el constructor con argument 10. Luego obj1.calculate() ejecuta e imprime el método sin argumentos. Luego obj1.calculate(5) ejecuta e imprime el método con argumento entero 5. La instancia obj2 cambia, ésta ejecuta e imprime el constructor con int argument 20 Aquí se repite igual que en la 1, primero el método sin argumentos y luego con argumento enter. Al final queda la a).

Pregunta 20

Incorrecta

Se puntúa 0,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

¿Cuál será el resultado del siguiente fragmento de código?

```
public class OverloadedConstructorsTest {
    private String name;
    private int age;
    public OverloadedConstructorsTest() {
        this("Unknown", 0);
        System.out.println("Default constructor called");
    }
    public OverloadedConstructorsTest(String name) {
        this(name, 0);
        System.out.println("Constructor with name called");
    }
    public OverloadedConstructorsTest(String name, int age) {
        this.name = name;
        this.age = age;
        System.out.println("Constructor with name and age called");
    }
    public static void main(String[] args) {
    OverloadedConstructorsTest obj1 = new OverloadedConstructorsTest();
    OverloadedConstructorsTest obj2 = new OverloadedConstructorsTest("Alice");
    OverloadedConstructorsTest obj3 = new OverloadedConstructorsTest("Bob", 30);
    }
}
```

Seleccione una:

a.
```
Default constructor called
Constructor with name and age called
Constructor with name called
Constructor with name and age called
Constructor with name and age called
```

b.
```
Constructor with name and age called
Default constructor called
Constructor with name called
Constructor with name and age called
```

c.
```
Constructor with name and age called
Default constructor called
Constructor with name and age called
Constructor with name called
Constructor with name and age called
```

d.
```
Default constructor called
Constructor with name called
Constructor with name and age called
```

Justificación: this(...) debe ser la primera sentencia y se ejecuta completo antes que el resto del constructor que lo invoca, así que el mensaje del constructor (String, int) sale antes que el del constructor por defecto y el del constructor (String). Son 5 líneas en total. La opción a es tentadora si se piensa que el println propio se ejecuta antes de la delegación.

Fuente: Java.pdf #150

La respuesta correcta es:

```
Constructor with name and age called
Default constructor called
Constructor with name and age called
Constructor with name called
Constructor with name and age called
```

Mi justificación
En orden de ejecución en el main, se ejecuta el constructor default, el constructor con nombre y el constructor con nombre y edad en ese orden. Los que tienen los textos de estos constructores es la d)



Pregunta 21

Incorrecta

Se puntúa 0.00 sobre 1.00

Marcar pregunta

Nivel: **Avanzado**

Dado el siguiente código ¿cuál es el resultado?

```
public class Recursividad {
    public Recursividad() {
        this(10);
        System.out.println("Default");
    }
    public Recursividad(int val) {
        this();
        System.out.println("Numerico");
    }
    public static void main(String[] args) {
        new Recursividad();
    }
}
```

Seleccione una:

a. Error de compilación
b. Lanza StackOverflowError ⊗
c. Imprime Numerico y luego Default
d. Imprime Default y luego Numerico

Justificación: Los constructores se llaman entre sí con this(10) y this(), y javac detecta esa invocación recursiva al compilar (JLS 8.8.7). El distractor más tentador es StackOverflowError, que sería el resultado si el compilador no la detectara, pero el programa nunca llega a ejecutarse.

Fuente: Quiz APX #9

La respuesta correcta es: Error de compilación

Mi justificación
El constructor recursividad() llama al constructor recursividad(10), el cual llama al constructor recursividad() y así sucesivamente creando un ciclo infinito. Esto lanza un error de stackoverflowError ya que eventualmente nos quedaremos sin memoria.



Pregunta **22**

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Avanzado**

What will be the result of attempting to compile and run the following code snippet?

```
public class RecursiveConstructorTest {
    private int value;

    public RecursiveConstructorTest() {
        this(10);
        System.out.println("No-argument constructor called");
    }

    public RecursiveConstructorTest(int value) {
        this();
        this.value = value;
        System.out.println("Constructor with int argument called: " + value);
    }

    public static void main(String[] args) {
        RecursiveConstructorTest obj = new RecursiveConstructorTest();
    }
}
```

Seleccione una:

a. Compilation error due to recursive constructor calls. ✓

b. Runtime error due to infinite recursion.

c. No-argument constructor called. Constructor with int argument called: 10.

d. Constructor with int argument called: 10. No-argument constructor called.

Justificación: javac detecta el ciclo de llamadas this() entre los dos constructores y rechaza el código con 'recursive constructor invocation' (JLS 8.8.7). El distractor tentador es un error de recursión infinita en ejecución, que sí ocurriría con métodos recursivos, pero con constructores el compilador lo impide antes.

Fuente: Java.pdf #202 · también aparece en: Cuestionario APX V3 #10

La respuesta correcta es: Compilation error due to recursive constructor calls.

Mi justificación
El constructor recursividad() llama al constructor recursividad(10), el cual llama al constructor recursividad() y así sucesivamente creando un ciclo infinito de constructores recursivos.

Pregunta 23

Incorrecta

Se puntúa 0,00 sobre 1,00

Marcar pregunta

Nivel: **Avanzado**

Give What is the result?

```
public class Test {
    public static void main(String[] args) {
        float f = 3.14f;
        new Test().doIt(f);
    }
    void doIt(Float f) {
        System.out.println("Float");
    }
    void doIt(double d) {
        System.out.println("double");
    }
    void doIt(float... fs) {
        System.out.println("varargs");
    }
}
```

Seleccione una:

- a. Compilations fails.
- b. An exception is thrown at runtime.
- c. The output is not predictable.
- d. Float ✗
- e. varargs
- f. double

Justificación: La resolución de sobrecarga busca primero un método aplicable sin boxing ni varargs. float -> double (ampliación primitiva) ya es aplicable en esa fase, así que gana doIt(double). El boxing a Float solo se considera en la segunda fase y los varargs en la tercera.

Fuente: Java.pdf #81 · también aparece en: Simulación APX V4 #26

La respuesta correcta es: double

Mi justificación
Bastante directa, doIt(f) llamaría al método que ejecuta doIt(float f) e imprime “float”.



Pregunta 24

Incorrecta

Se puntúa 0,00 sobre 1,00

Marcar pregunta

Nivel: **Avanzado**

¿Cuántos de los fragmentos de código, si compilan?

```
public static void main(String...a) {
public static void main(String. a) {
public static void main(String... a) {
public static void main(String[]... a) {
public static void main(String...[] a) {
```

Seleccione una:

a. 2
b. 3
c. Todos
d. Ninguno

Justificación: Compilan tres. String...a y String... a son varargs (el espacio no importa), y String[]... a es un varargs de String[] (equivale a String[][]): compila, aunque la JVM no lo acepta como punto de entrada. String. a (un solo punto) y String...[] a (corchetes después de los puntos) son errores de sintaxis. El distractor '2' sale de pensar que String[]... no compila porque no es un main válido.

Fuente: Quiz Desarrollo APX 1 #7

La respuesta correcta es: 3

Mi justificación
La primera es inválida ya que necesita un espacio después de los 3 puntos. La segunda es inválida ya que String. así con un punto es un error de sintaxis. El tercero es válido. El cuarto es válido. El quinto es inválido ya que String... [] es un error de sintaxis.

RESULTADO = 13/24 = 5.4